

RV64 Instruction Set — Decoding Reference

Base ISA (RV64I) — R / I / S / B / U / J types. Excludes M (multiply/divide) and C (compressed) extensions.

opcode	rd	funct3	rs1	rs2	funct7	immediate	[RV64] = RV64I-only instruction
--------	----	--------	-----	-----	--------	-----------	---------------------------------

Mnemonic	Fmt	opcode	funct3	funct7 / imm	Bit layout [31:0]						Operation
R-Type — Register–Register (opcode 0110011 / 0111011 for *W variants)											
ADD	R	0110011	000	0000000	0000000	rs2	rs1	000	rd	0110011	rd = rs1 + rs2
SUB	R	0110011	000	0100000	0100000	rs2	rs1	000	rd	0110011	rd = rs1 - rs2
AND	R	0110011	111	0000000	0000000	rs2	rs1	111	rd	0110011	rd = rs1 & rs2
OR	R	0110011	110	0000000	0000000	rs2	rs1	110	rd	0110011	rd = rs1 rs2
XOR	R	0110011	100	0000000	0000000	rs2	rs1	100	rd	0110011	rd = rs1 ^ rs2
SLL	R	0110011	001	0000000	0000000	rs2	rs1	001	rd	0110011	rd = rs1 << rs2[5:0]
SRL	R	0110011	101	0000000	0000000	rs2	rs1	101	rd	0110011	rd = rs1 >> rs2[5:0] (logical)
SRA	R	0110011	101	0100000	0100000	rs2	rs1	101	rd	0110011	rd = rs1 >> rs2[5:0] (arithmetic)
SLT	R	0110011	010	0000000	0000000	rs2	rs1	010	rd	0110011	rd = (rs1 < rs2) ? 1:0 (signed)
SLTU	R	0110011	011	0000000	0000000	rs2	rs1	011	rd	0110011	rd = (rs1 < rs2) ? 1:0 (unsigned)
ADDW	R	0111011	000	0000000	0000000	rs2	rs1	000	rd	0111011	rd = sext(rs1[31:0] + rs2[31:0]) [RV64]
SUBW	R	0111011	000	0100000	0100000	rs2	rs1	000	rd	0111011	rd = sext(rs1[31:0] - rs2[31:0]) [RV64]
SLLW	R	0111011	001	0000000	0000000	rs2	rs1	001	rd	0111011	rd = sext(rs1[31:0] << rs2[4:0]) [RV64]
SRLW	R	0111011	101	0000000	0000000	rs2	rs1	101	rd	0111011	rd = sext(rs1[31:0] >> rs2[4:0]) (logical) [RV64]
SRAW	R	0111011	101	0100000	0100000	rs2	rs1	101	rd	0111011	rd = sext(rs1[31:0] >> rs2[4:0]) (arith) [RV64]
I-Type — Immediate Arithmetic (opcode 0010011)											
ADDI	I	0010011	000	imm	imm[11:0]	rs1	000	rd	0010011		rd = rs1 + sext(imm)
ANDI	I	0010011	111	imm	imm[11:0]	rs1	111	rd	0010011		rd = rs1 & sext(imm)
ORI	I	0010011	110	imm	imm[11:0]	rs1	110	rd	0010011		rd = rs1 sext(imm)
XORI	I	0010011	100	imm	imm[11:0]	rs1	100	rd	0010011		rd = rs1 ^ sext(imm)
SLTI	I	0010011	010	imm	imm[11:0]	rs1	010	rd	0010011		rd = (rs1 < sext(imm)) ? 1:0 (signed)
SLTIU	I	0010011	011	imm	imm[11:0]	rs1	011	rd	0010011		rd = (rs1 < sext(imm)) ? 1:0 (unsigned)
SLLI	I	0010011	001	000000 shamt[5:0]	000000	shamt[5:0]	rs1	001	rd	0010011	rd = rs1 << shamt[5:0]

Mnemonic	Fmt	opcode	funct3	funct7 / imm	Bit layout [31:0]						Operation
SRLI	I	0010011	101	000000 shamt[5:0]	000000	shamt[5:0]	rs1	101	rd	0010011	rd = rs1 >> shamt[5:0] (logical)
SRAI	I	0010011	101	010000 shamt[5:0]	010000	shamt[5:0]	rs1	101	rd	0010011	rd = rs1 >> shamt[5:0] (arithmetic)
I-Type — Loads (opcode 0000011)											
LB	I	0000011	000	offset	imm[11:0]	rs1	000	rd	0000011		rd = sext(M[rs1+sext(imm)][7:0])
LBU	I	0000011	100	offset	imm[11:0]	rs1	100	rd	0000011		rd = zext(M[rs1+sext(imm)][7:0])
LH	I	0000011	001	offset	imm[11:0]	rs1	001	rd	0000011		rd = sext(M[rs1+sext(imm)][15:0])
LHU	I	0000011	101	offset	imm[11:0]	rs1	101	rd	0000011		rd = zext(M[rs1+sext(imm)][15:0])
LW	I	0000011	010	offset	imm[11:0]	rs1	010	rd	0000011		rd = sext(M[rs1+sext(imm)][31:0])
LWU	I	0000011	110	offset	imm[11:0]	rs1	110	rd	0000011		rd = zext(M[rs1+sext(imm)][31:0]) [RV64]
LD	I	0000011	011	offset	imm[11:0]	rs1	011	rd	0000011		rd = M[rs1+sext(imm)][63:0] [RV64]
I-Type — Jump & System (various opcodes)											
JALR	I	1100111	000	offset	imm[11:0]	rs1	000	rd	1100111		rd=PC+4 ; PC=(rs1+sext(imm))&~1
ECALL	I	1110011	000	000000000000	000000000000	00000	000	00000	1110011		Environment call (syscall)
EBREAK	I	1110011	000	000000000001	000000000001	00000	000	00000	1110011		Breakpoint trap
I-Type — RV64 Word Immediate (opcode 0011011)											
ADDIW	I	0011011	000	imm	imm[11:0]	rs1	000	rd	0011011		rd = sext(rs1[31:0] + sext(imm)) [RV64]
SLLIW	I	0011011	001	0000000 shamt[4:0]	0000000	shamt[4:0]	rs1	001	rd	0011011	rd = sext(rs1[31:0] << shamt[4:0]) [RV64]
SRLIW	I	0011011	101	0000000 shamt[4:0]	0000000	shamt[4:0]	rs1	101	rd	0011011	rd = sext(rs1[31:0] >> shamt[4:0]) (logical) [RV64]
SRAIW	I	0011011	101	0100000 shamt[4:0]	0100000	shamt[4:0]	rs1	101	rd	0011011	rd = sext(rs1[31:0] >> shamt[4:0]) (arith) [RV64]
S-Type — Stores (opcode 0100011)											
SB	S	0100011	000	[11:5] [4:0]	imm[11:5]	rs2	rs1	f3	imm[4:0]	opcode	M[rs1+sext(imm)] = rs2[7:0]
SH	S	0100011	001	[11:5] [4:0]	imm[11:5]	rs2	rs1	f3	imm[4:0]	opcode	M[rs1+sext(imm)] = rs2[15:0]
SW	S	0100011	010	[11:5] [4:0]	imm[11:5]	rs2	rs1	f3	imm[4:0]	opcode	M[rs1+sext(imm)] = rs2[31:0]
SD	S	0100011	011	[11:5] [4:0]	imm[11:5]	rs2	rs1	f3	imm[4:0]	opcode	M[rs1+sext(imm)] = rs2[63:0] [RV64]
B-Type — Branches (opcode 1100011) — imm encodes signed byte offset, always x2											
BEQ	B	1100011	000	[12 10:5 4:1 11]	im[12 10:5]	rs2	rs1	f3	im[4:1 11]	opcode	if rs1==rs2: PC += sext(imm<<1)
BNE	B	1100011	001	[12 10:5 4:1 11]	im[12 10:5]	rs2	rs1	f3	im[4:1 11]	opcode	if rs1!=rs2: PC += sext(imm<<1)

Mnemonic	Fmt	opcode	funct3	funct7 / imm	Bit layout [31:0]						Operation
BLT	B	1100011	100	[12 10:5 4:1 11]	im[12 10:5]	rs2	rs1	f3	im[4:1 11]	opcode	if rs1<rs2 (signed): PC += sext(imm<<1)
BLTU	B	1100011	110	[12 10:5 4:1 11]	im[12 10:5]	rs2	rs1	f3	im[4:1 11]	opcode	if rs1<rs2 (unsigned): PC += sext(imm<<1)
BGE	B	1100011	101	[12 10:5 4:1 11]	im[12 10:5]	rs2	rs1	f3	im[4:1 11]	opcode	if rs1>=rs2 (signed): PC += sext(imm<<1)
BGEU	B	1100011	111	[12 10:5 4:1 11]	im[12 10:5]	rs2	rs1	f3	im[4:1 11]	opcode	if rs1>=rs2 (unsigned): PC += sext(imm<<1)
U-Type — Upper Immediate											
LUI	U	0110111	—	imm[31:12]	imm[31:12]		rd		opcode		rd = imm << 12 (loads upper 20 bits)
AUIPC	U	0010111	—	imm[31:12]	imm[31:12]		rd		opcode		rd = PC + (imm << 12)
J-Type — Unconditional Jump											
JAL	J	1101111	—	[20 10:1 11 19:12]	imm[20 10:1 11 19:12]		rd		opcode		rd = PC+4 ; PC += sext(imm<<1) [20-bit signed offset, x2]